МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Потеха Вячеслав Семёнович
(подпись) (фамилия, имя, отчество)

Группы ИПо-42

Руководитель практической подготовки от профильной организации

_____
(должность)

_____ _____
(подпись) (фамилия, имя, отчество)

МП «16» марта 2026 года

Руководитель практической подготовки от института

_____ _____
(подпись) (фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

Содержание

Введение 2
Раздел 1. Организационно подготовительный этап прохождения практики на предприятии 5
Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии 5
Раздел 2. Ознакомление со структурой и характером деятельности подразделения 8
Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении 8
Тема 2.2. Ознакомление с должностными и функциональными обязанностями 12
Тема 2.3. Организация информационного обеспечения и защиты информации предприятия 14
Тема 2.4. Структурные и функциональные схемы предприятия 18
Раздел 3. Работа на рабочих местах или в подразделениях предприятиях 22
Тема 3.1. Выполнение производственных заданий 22
Заключение 39
Список используемых источников 40
Приложения 42

Введение

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Производственная практика направлена на освоение обучающимися профессиональных компетенций в процессе самостоятельного выполнения определенных видов работ, необходимых для осуществления профессиональной деятельности в максимально приближенных к производству условиях, в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

В настоящей работе представлен отчет о прохождении производственной практики студента-практиканта. Основными видами деятельности которой являлись: в ходе производственной практики мною выполнялись работы по формированию требований к программным модулям на основе анализа проектной и технической документации. Целью данной деятельности являлось обеспечение корректного взаимодействия компонентов разрабатываемого программного обеспечения. В процессе работы изучались функциональные и нефункциональные требования, архитектурные решения и спецификации, что позволило сформировать непротиворечивые и реализуемые требования к отдельным модулям системы.

Кроме того, в рамках практики осуществлялось описание процесса моделирования разработки программного обеспечения. Были рассмотрены и задокументированы основные этапы жизненного цикла разработки, применяемые методологии, а также инструменты и подходы, используемые

для создания моделей программных продуктов. Полученные результаты позволяют наглядно представить последовательность и взаимосвязь процессов при разработке ПО в условиях реального производства.

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Выполнять интеграцию модулей в программное обеспечение;
2. Выполнять отладку программного модуля с использованием специализированных программных средств;
3. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения;
4. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;
5. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;
6. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности
7. Планировать и реализовывать собственное профессиональное и личностное развитие;
8. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами
9. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.
10. Использовать информационные технологии в профессиональной деятельности
11. Пользоваться профессиональной документацией на государственном и иностранном языках

Раздел 1. Организационно подготовительный этап прохождения практики на предприятии

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

В период прохождения производственной практики мною неукоснительно соблюдались правила внутреннего трудового распорядка предприятия, требования охраны труда и техники безопасности. Все задания руководителя практики выполнялись в установленные сроки с соблюдением последовательности технологических операций, предусмотренных регламентом работы. О возникающих трудностях и нештатных ситуациях я своевременно информировал руководителя. Ведение дневника практики осуществлялось регулярно с фиксацией выполненных работ и приобретённых навыков. Конфиденциальность информации, полученной в ходе работы, соблюдалась. На рабочем месте поддерживались чистота и порядок.

Инструктажи и подготовка к работе. Перед допуском к самостоятельной работе мною были пройдены вводный инструктаж по охране труда и первичный инструктаж на рабочем месте. В ходе инструктажей изучены: знаки безопасности и предупредительные таблички, места расположения аптечки первой помощи и средств пожаротушения, пути эвакуации и номера экстренных служб. Также были разъяснены правила использования средств индивидуальной защиты (СИЗ) при выполнении специфических работ.

Проверка оборудования перед началом работы. Ежедневно перед началом работы я выполнял осмотр рабочего места и оборудования:

- проверял целостность изоляции проводов, штепсельных вилок, розеток и корпусов приборов;

- убеждался в отсутствии повреждений на корпусе прибора и оголённых токоведущих частей;

- контролировал наличие заземления (при необходимости);

- осматривал рабочее место на отсутствие влаги, горючих материалов и посторонних предметов;

- при необходимости надевал диэлектрические перчатки и другие требуемые СИЗ.

Соблюдение правил безопасности во время работы. В процессе выполнения производственных задач я строго руководствовался следующими правилами:

- включение приборов осуществлялось только сухими руками с удержанием за корпус вилки;

- отключение от сети производилось аккуратно за корпус вилки, без натяжения шнура;

- не допускалось перегрузка электросети и использование повреждённых удлинителей;

- ремонт электроприборов и вскрытие корпусов под напряжением не производились;

- исключался контакт с работающими приборами мокрыми руками или влажными предметами;

- включённое оборудование не оставалось без надзора;

- при работе с измерительными приборами соблюдалась инструкция по эксплуатации, не превышались допустимые пределы измерений;

- при замене элементов питания соблюдалась полярность и использовались только рекомендованные батареи.

Действия в аварийных ситуациях. Мною были изучены и отработаны алгоритмы действий при возникновении нештатных ситуаций:

- при появлении запаха гари, искрении или перегреве прибора — немедленное отключение от сети;

- при поражении человека электрическим током — отключение питания, оказание первой помощи, вызов медицинского персонала;

- при возгорании электроприбора — отключение питания, использование огнетушителя (не водой!), вызов пожарной службы;

- при обнаружении обрыва проводов или повреждения электрооборудования — не приближаться, оповестить ответственного за электробезопасность.

Завершение работ. По окончании рабочего дня мною выполнялись следующие действия:

- отключение приборов от сети;

- уборка кабелей и проводов, исключение их нахождения в проходах;

- проверка отсутствия нагрева корпусов и посторонних запахов;

-сдача оборудования на хранение в установленное место.

Таким образом, за время прохождения производственной практики требования охраны труда и техники безопасности соблюдались в полном объеме, нарушений и несчастных случаев не допущено.

Раздел 2. Ознакомление со структурой и характером деятельности подразделения

Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении

2.1.1 Структура предприятия и ИТ-подразделения.

ООО «АРУС ПАРУС» является обществом с ограниченной ответственностью, зарегистрированным 08.08.2024 в г. Перми. Организационная структура предприятия относится к линейно-функциональному типу, что характерно для небольших компаний в сфере разработки программного обеспечения.

Высшим органом управления является Общее собрание участников. В состав учредителей входят три физических лица с распределением долей:

- Сабуров Алексей Леонидович — 34% уставного капитала;
- Карпов Сергей Владимирович — 33% уставного капитала;
- Харгелия Олег Резович — 33% уставного капитала.

Единоличным исполнительным органом выступает генеральный директор Харгелия Олег Резович, который осуществляет руководство текущей деятельностью компании. Таблица 2 - отражает функциональную структуру предприятия и распределение зон ответственности между основными подразделениями. Она демонстрирует, какие задачи закреплены за каждой функциональной единицей, как соотносятся их обязанности с зарегистрированными видами деятельности по ОКВЭД, и позволяет проследить логику взаимодействия подразделений в рамках производственного цикла компании. Данная структура обеспечивает прозрачность управления и эффективное распределение трудовых ресурсов при реализации проектов в сфере промышленной автоматизации.

Таблица 1 – функции персонала (сотрудники).

Функциональное направление	Основные функции
Административно-управленческий персонал	Общее руководство, стратегическое планирование, взаимодействие с заказчиками, финансовое и кадровое обеспечение
Проектный отдел (разработка ПО)	Разработка компьютерного программного обеспечения (основной вид деятельности по ОКВЭД 62.01)
Отдел консультирования и сопровождения	Консультативные услуги в области компьютерных технологий (ОКВЭД 62.02), управление компьютерным оборудованием (ОКВЭД 62.03)
Отдел обработки данных	Деятельность по обработке данных, предоставление услуг по размещению информации (ОКВЭД 63.11), деятельность web-порталов (ОКВЭД 63.12)

Ввиду небольшого масштаба организации (микропредприятие, включено в реестр МСП 10.09.2024), выделенного отдельного ИТ-подразделения не предусмотрено. Функции разработки, сопровождения и администрирования информационных систем распределены между проектными командами, формируемыми под конкретные задачи заказчиков.

2.1.2 Функции подразделений и их взаимодействие

В структуре предприятия можно выделить следующие основные функциональные единицы:

Взаимодействие между функциональными направлениями осуществляется следующим образом:

- Административно-управленческий персонал получает заказы от заказчиков и распределяет задачи между проектными группами;
- Проектный отдел разрабатывает программное обеспечение и передаёт его в отдел сопровождения для внедрения и поддержки;
- Отдел консультирования обеспечивает обратную связь с заказчиками и формирует требования для доработки продуктов;
- Отдел обработки данных обеспечивает техническую инфраструктуру для работы всех подразделений.

Компания также имеет устойчивые деловые связи с профильными организациями через своих учредителей (ООО «АРУС ТЕХНОЛОГИЯ», ООО «ФАРРО-ТМ», ООО «НПП «АКВАСИНТ»), что позволяет привлекать дополнительные ресурсы и экспертизу при реализации комплексных проектов в области промышленной автоматизации.

2.1.3 Режим работы и формы организации труда

Режим работы предприятия:

- Стандартная рабочая неделя: 5-дневная с понедельника по пятницу;
- Продолжительность рабочего дня: 8 часов (с 9:00 до 18:00);
-Обеденный перерыв: 1 час (с 13:00 до 14:00);
- Выходные дни: суббота, воскресенье.

Формы организации труда:

- Для инженерно-технического состава предусмотрен гибкий график работы с возможностью корректировки времени начала и окончания рабочего дня (в пределах 2 часов) при условии отработки суммарного количества рабочих часов;
- Применяется проектная форма организации труда — сотрудники объединяются во временные проектные группы для выполнения конкретных задач заказчиков;
- Используется удалённый режим работы (частично или полностью) для отдельных категорий сотрудников, занятых разработкой программного обеспечения;
- Ведётся электронный учёт рабочего времени через корпоративную систему табельного учёта;
- Регулярно проводятся оперативные совещания (планёрки) для координации действий проектных групп.

2.1.4 Краткая характеристика предприятия.

Таблица 2 – Краткая характеристика предприятия.

Параметр	Значение

Параметр	Значение
Полное наименование	Общество с ограниченной ответственностью «АРУС ПАРУС»
Сокращённое наименование	ООО «АРУС ПАРУС»
Организационно-правовая форма	Общество с ограниченной ответственностью
Дата регистрации	08.08.2024
Юридический адрес	614051, Пермский край, г. Пермь, ул. Пушкарская, д. 98, кв. 75
ИНН/КПП	5906181316 / 590601001
ОГРН	1245900013226
Уставный капитал	100 000 руб.
Основной вид деятельности	Разработка компьютерного программного обеспечения (ОКВЭД 62.01)
Дополнительные виды деятельности	Торговля оптовая программным обеспечением; консультативные услуги в области компьютерных технологий; управление компьютерным оборудованием; обработка данных; деятельность web-порталов; дополнительное профессиональное образование
Руководитель	Генеральный директор Харгелия Олег Резович
Категория субъекта МСП	Микропредприятие (с 10.09.2024)
Система налогообложения	УСН (Упрощённая система налогообложения)
Телефон	+7 (342) 214-00-99, +7 916 924-06-86

Основные направления деятельности:

Компания специализируется на разработке и внедрении программного обеспечения, систем автоматизации технологических процессов (АСУ ТП), а также предоставлении консультационных услуг в области информационных технологий. Особое внимание уделяется промышленной автоматизации и созданию цифровых решений для производственных предприятий. С апреля

2025 года компания является членом Ассоциации проектировщиков «Развитие проектной отрасли», что подтверждает её профессиональный статус в области проектирования и позволяет выполнять работы на объектах капитального строительства, включая особо опасные и технически сложные объекты.

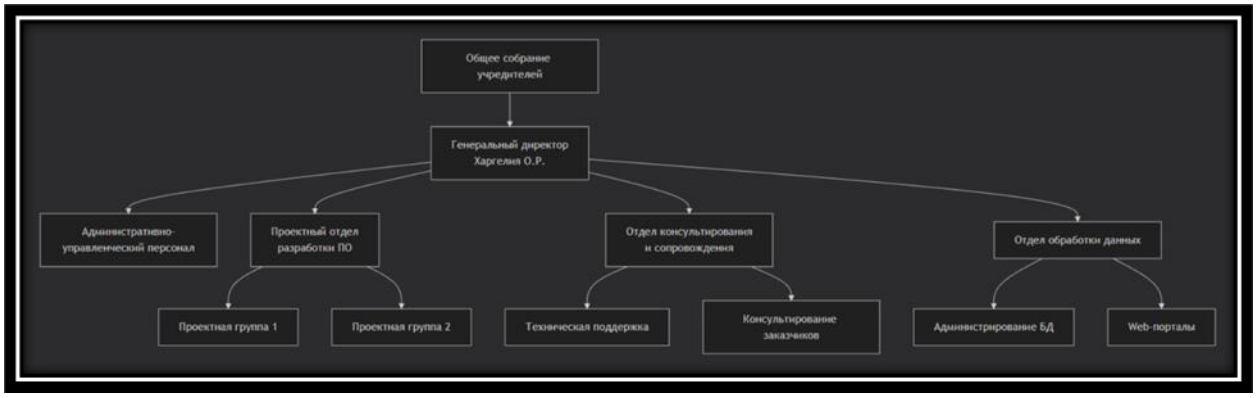

Рисунок 1 – Организационная структура ООО «АРУС ПАРУС»

Тема 2.2. Ознакомление с должностными и функциональными обязанностями

2.2.1 Изучение должностной инструкции сотрудника ИТ-подразделения.[2]

В ходе практики изучена должностная инструкция инженера-программиста (специалиста по АСУ ТП) ООО «АРУС ПАРУС».

Общие положения:

- должность относится к категории специалистов;

- назначается лицо с высшим техническим образованием без требований к стажу или со средним профессиональным с опытом работы;

- подчиняется руководителю проектного отдела;

-назначается и освобождается от должности приказом генерального директора.

Квалификационные требования:

Специалист должен знать:

- нормативные и методические материалы по автоматизации;

- средства вычислительной техники и связи;

- порядок разработки технической документации;

- правила эксплуатации средств автоматизации;

- методы оценки эффективности внедрения автоматизации;

- правила охраны труда.

2.2.2 Права и обязанности специалиста

Таблица 3 - Должностные обязанности.

Направление	Обязанности
Разработка ПО	Разработка решений по ТЗ; создание алгоритмов; программирование ПЛК (стандарт МЭК 61131-3); разработка HMI/SCADA; тестирование и отладка
Сопровождение	Поддержка проектов; устранение отказов; техобслуживание ПО; доработка по требованиям заказчиков
Документирование	Разработка техдокументации; составление инструкций; ведение проектной отчётности
Работа с оборудованием	Чтение электрических схем; подбор оборудования; участие в пусконаладке

Права специалиста:

- знакомиться с проектами решений руководства;

- вносить предложения по совершенствованию работы;

- запрашивать информацию через руководителя;

-привлекать специалистов других подразделений;

- требовать условий для выполнения обязанностей.

2.2.3 Согласование индивидуального задания

С руководителем практики от ООО «АРУС ПАРУС» согласовано индивидуальное задание:

1. Участие в сопровождении ПО для автоматизации;

2. Ознакомление со SCADA-системами и средами разработки ПЛК;

3. Тестирование и отладка программных модулей;

4. Работа с технической документацией;

5. Сбор материалов для отчёта.

Сроки и форма отчётности утверждены в дневнике практики.

2.2.4 Цели и задачи практики

Цель: Получение практического опыта сопровождения ПО и изучение организации информационных процессов в сфере промышленной автоматизации.

Задачи:

1. Изучение структуры предприятия и работы отдела АСУ ТП;

2. Анализ аппаратных и программных средств автоматизации;

3. Участие в настройке специализированного ПО (SCADA, среды разработки);

4. Работа с базами данных технологических параметров;

5. Изучение методов защиты информации в промышленных сетях;

6. Сбор материалов для отчёта.

Ожидаемые результаты:

- закрепление теоретических знаний;

- формирование практических навыков работы с профессиональным ПО;

- приобретение опыта проектной работы;

- развитие профессиональных компетенций.

Тема 2.3. Организация информационного обеспечения и защиты информации предприятия

ООО «АРУС ПАРУС» как компания, специализирующаяся на разработке программного обеспечения (ОКВЭД 62.01), использует в своей деятельности широкий спектр программных средств, которые можно классифицировать по следующим направлениям:

1. Средства разработки и программирования:

- Среды разработки ПЛК: CODESYS V3.5, TIA Portal (Step 7, WinCC), Unity Pro (Control Expert) для программирования промышленных контроллеров;

- SCADA-системы: MasterSCADA 4D, Alpha.Platform, Simatic WinCC Professional для разработки человеко-машинного интерфейса и систем диспетчерского управления;

- Универсальные среды разработки: Visual Studio Code, Visual Studio Community для разработки вспомогательного ПО и скриптов.

2. Системы автоматизированного проектирования (САПР):

- sPlan, AutoCAD Electrical, EPLAN Electric P8 для разработки принципиальных электрических схем, кабельных журналов и компоновки щитов автоматики.

3. Системы управления базами данных:

- Microsoft SQL Server, PostgreSQL для работы с базами данных технологических параметров и архивации исторических данных.

4. Системы управления проектами и документацией:

- Git (система контроля версий) для хранения исходных кодов;

- Redmine / YouGile для управления проектными задачами;

- Корпоративная система электронного документооборота (СЭД).

5. Инструменты моделирования:

- AnyLogic, MATLAB/Simulink для моделирования и симуляции технологических процессов.

2.3.1 Процессы сбора, регистрации и обработки информации

Сбор информации:

- Данные с технологических объектов поступают через программируемые логические контроллеры (ПЛК), установленные на производственных площадках заказчиков;

- Информация с контрольно-измерительных приборов (КИП) собирается в реальном времени и передаётся по промышленным сетям;

- Техническая документация и требования формируются на основе взаимодействия с заказчиками и фиксируются в технических заданиях.

Регистрация информации:

- Технологические параметры регистрируются в специализированных базах данных (MS SQL Server, PostgreSQL) с привязкой ко времени;

- Исходные коды программ фиксируются в системе контроля версий Git;

- Проектная документация регистрируется в системе электронного документооборота с присвоением уникальных номеров и версий;

- Все изменения в проектах документируются с указанием ответственных лиц и дат.

Обработка информации:

- SCADA-системы осуществляют визуализацию технологических параметров в реальном времени;

- Данные анализируются для выявления отклонений и нештатных ситуаций;

- Формируются отчёты и аналитические сводки для заказчиков;

- Информация архивируется для долговременного хранения и последующего анализа.

2.3.2 Методология проектирования и эксплуатации информационных систем.[3]

Методология проектирования:

В ООО «АРУС ПАРУС» применяется классический каскадный подход с элементами итеративной разработки:

1. Анализ требований – сбор и формализация требований заказчика, разработка технического задания;

2. Проектирование – создание архитектуры системы, разработка алгоритмов, выбор оборудования;

3. Разработка – программирование ПЛК и SCADA-систем, создание интерфейсов;

4. Тестирование – отладка программных модулей, проверка на тестовых стендах;

5. Внедрение – установка ПО на объекте заказчика, пусконаладочные работы;

6. Сопровождение – техническая поддержка, доработка по требованиям заказчиков.

Методология эксплуатации:

- Мониторинг – непрерывный контроль работоспособности систем;
- Регламентное обслуживание – плановые проверки и обновления;
- Управление инцидентами – фиксация и устранение отказов;
- Управление изменениями – контроль версий и документирование доработок.

2.3.3 Методы защиты информации.

В ООО «АРУС ПАРУС» применяются следующие методы защиты информации, соответствующие требованиям к предприятиям, работающим с промышленными системами и данными заказчиков:

Организационные меры:

- Разграничение прав доступа к информационным ресурсам на основе ролевой модели;
- Политика конфиденциальности и защиты коммерческой тайны;
- Инструктаж сотрудников по вопросам информационной безопасности;
- Регламентированный порядок работы с проектной документацией.

Таблица 4 - Технические средства защиты.

Уровень защиты	Применяемые средства
Защита рабочих станций	Многоуровневая антивирусная защита Kaspersky Endpoint Security с централизованным управлением, регулярное обновление антивирусных баз, контроль запуска программ
Сетевая защита	Межсетевые экраны, фильтрация трафика, корпоративный прокси-сервер
Защита доступа	Разграничение прав через доменные службы Active Directory, индивидуальные логины и пароли, учётные записи с различными уровнями привилегий

Уровень защиты	Применяемые средства
Защита от перебоев электропитания	Источники бесперебойного питания APC Smart-UPS с двойным преобразованием энергии
Резервное копирование	Системы резервного копирования (Veeam Backup, Acronis True Image), RAID-массивы на файловых серверах
Удалённый доступ	VPN-соединения для защищённого доступа сотрудников к корпоративным ресурсам

Программно-технические меры:

- Централизованное управление обновлениями безопасности через корпоративный сервер обновлений (WSUS);
- Использование лицензионного программного обеспечения;
- Ведение логов и журналирование событий;
- Контроль целостности систем.

Правовые меры:

- Соблюдение требований законодательства РФ в области защиты информации;
- Использование лицензионного ПО с подтверждёнными правами;
- Договорные обязательства с заказчиками о конфиденциальности.

Тема 2.4. Структурные и функциональные схемы предприятия

2.4.1 Требования к техническим и программным средствам

Таблица 5 - Требования к аппаратному обеспечению рабочих мест.

Компонент	Минимальные требования	Фактическое оснащение
Процессор	Intel Core i5 / AMD Ryzen 5	Intel Core i7-12700
ОЗУ	16 ГБ DDR4	32 ГБ DDR4
Накопитель	SSD 256 ГБ	NVMe SSD 1 ТБ
Видеокарта	Дискретная, 2 ГБ	NVIDIA GeForce RTX 3060

Компонент	Минимальные требования	Фактическое оснащение
Мониторы	1 монитор 22"	2 монитора 27" IPS
Сеть	100 Мбит/с	Гигабитное подключение
ИБП	-	APC Smart-UPS 1500 ВА

Таблица 6 - Требования к программному обеспечению.

Категория ПО	Используемое ПО
Операционная система	Windows 11 Pro for Workstations
Офисный пакет	Microsoft Office Professional Plus
Средства разработки ПЛК	CODESYS V3.5, TIA Portal, Unity Pro
SCADA-системы	MasterSCADA 4D, Alpha.Platform, Simatic WinCC
САПР	sPlan, AutoCAD Electrical, EPLAN Electric P8
СУБД	MS SQL Server, PostgreSQL
Антивирус	Kaspersky Endpoint Security
Резервное копирование	Veeam Backup, Acronis True Image

2.4.2 Анализ используемых средств защиты информации

Организационные средства:

- Ролевая модель доступа;
- Политика конфиденциальности;
- Инструктаж сотрудников;
- Договоры о неразглашении.

Программно-технические средства — это совокупность аппаратного обеспечения (компьютеры, серверы, сетевое оборудование) и установленного на нем программного обеспечения, совместно обеспечивающих выполнение задач информационной системы. Простыми словами: «железо» + софт,

работающие как единое целое для сбора, обработки, хранения и передачи информации. (см. Таблица 8)

Таблица 7 - Программно-технические средства.

Уровень защиты	Средства защиты
Сетевая защита	Межсетевые экраны, прокси-сервер, фильтрация трафика, VPN
Защита рабочих станций	Kaspersky Endpoint Security, контроль запуска программ, WSUS
Защита данных	Резервное копирование, RAID-массивы, шифрование
Контроль доступа	Active Directory, индивидуальные логины/пароли

Инженерно-технические средства:

- ИБП с двойным преобразованием;
- Сетевые фильтры с подавлением помех;
- Защитное заземление;
- Контроль состояния кабельных трасс.

2.4.3 Ознакомление с делопроизводством и ИТ-документацией

Таблица 8 - Виды документации.

Тип документации	Форма ведения
Техническая документация	СЭД, бумажный архив
Проектная документация (ТЗ, чертежи)	СЭД, файлы САПР
Программная документация	Git, файловая система
Эксплуатационная документация	Электронная (PDF), бумажная
Отчётная документация	СЭД, печатные формы
Организационно-распорядительная	Электронная, бумажная

Порядок ведения:

- Регистрация с уникальными номерами;
- Хранение в электронном виде с резервированием;
- Учёт версий в Git;
- Разграничение прав доступа;
- Архивация завершённых проектов.

2.4.4 Применяемые технологии безопасности

Таблица 9 – Применяемые технологии безопасности.

Технология	Назначение	Реализация
Межсетевое экранирование	Защита от НСД извне	Корпоративный межсетевой экран
Прокси-сервер	Фильтрация трафика	Корпоративный прокси-сервер
VPN	Защищённый удалённый доступ	VPN-сервер
Антивирусная защита	Защита от вредоносного ПО	Kaspersky Endpoint Security
Централизованное обновление	Управление обновлениями	WSUS
Резервное копирование	Защита данных	Veeam Backup, Acronis
RAID-массивы	Отказоустойчивость	Файловые серверы
ИБП	Защита от перебоев питания	APC Smart-UPS
Политика паролей	Защита доступа	Требования к сложности паролей

Раздел 3. Работа на рабочих местах или в подразделениях предприятиях

Тема 3.1. Выполнение производственных заданий

3.1.1. использование выбранной системы контроля версий;

В ходе разработки программного обеспечения в период производственной практики применялась система контроля версий Git. Данный инструмент относится к классу распределенных систем управления версиями и предназначен для фиксации изменений в кодовой базе, организации коллективной работы над проектом, а также хранения всей истории разработки с возможностью восстановления предыдущих состояний.

Работа с Git включала выполнение следующих операций: инициализация локального хранилища, создание коммитов для сохранения промежуточных результатов, использование ветвления для изолированной разработки отдельных функций и синхронизация с удаленным репозиторием. Для обеспечения сохранности кода и предоставления доступа к проекту руководителю практики локальный репозиторий был интегрирован с удаленным репозиторием на платформе GitHub. Управление версиями осуществлялось как средствами графического интерфейса Visual Studio 2022, так и посредством командной строки при выполнении слияния веток и устранения конфликтов. Применение Git позволило обеспечить надлежащий контроль версий разрабатываемого программного продукта и соблюдение требований к организации командной работы.

3.1.2. разработка программного обеспечения;

При создании программного продукта применялась итерационная модель разработки. Выбор в пользу данной методологии обусловлен необходимостью постепенной реализации компонентов системы с возможностью корректировки на каждом этапе. Итерационный подход позволил последовательно разработать модули, отвечающие за ведение справочников партнеров, продукции и учета продаж. На каждой итерации

выполнялось тестирование созданного функционала и при необходимости вносились изменения с учетом уточненных требований. Гибкость итерационной модели, в отличие от линейной каскадной схемы, обеспечила возможность адаптации функциональных характеристик системы непосредственно в ходе выполнения работ.

Инструментальные средства разработки

В процессе создания программного продукта задействован следующий набор инструментов:

- Git и GitHub – распределенная система управления версиями и веб-платформа для хранения исходного кода и совместной работы;

- Visual Studio 2022 – основная интегрированная среда, используемая для написания, отладки и компиляции программного кода;

- PostgreSQL – реляционная СУБД, обеспечивающая хранение и обработку данных о партнерах, номенклатуре продукции и совершенных продажах;

- pgAdmin – графическое средство для администрирования и управления базой данных PostgreSQL;

- NuGet Package Manager – встроенный менеджер пакетов, предназначенный для подключения и обновления внешних библиотек в проекте.

Инструментальная среда разработки

Для реализации программного проекта применялась интегрированная среда Microsoft Visual Studio 2022, функционирующая на базе платформы .NET Framework 4.8. В качестве основного языка разработки выбран C#, построение графического интерфейса осуществлялось с использованием технологии WPF. Взаимодействие с данными обеспечивалось посредством Entity Framework Core — объектно-реляционного отображения, позволяющего выполнять запросы к базе данных PostgreSQL через LINQ и оперировать бизнес-сущностями в объектной парадигме.

Структурная модель приложения

В ходе проектирования разработана диаграмма классов, иллюстрирующая архитектуру программного продукта (рисунок 1). Ключевыми элементами модели являются:

- сущности предметной области (Partner, Product, Sale, PartnerType);
- классы-посредники (PartnerViewModel, SaleViewModel), обеспечивающие связь между данными и интерфейсом пользователя;
- контекст базы данных AppDbContext, отвечающий за управление соединениями и запросами.

На диаграмме отражены основные связи между компонентами, в частности отношение "один ко многим" между партнером и совершенными продажами, а также между типом партнера и списком партнеров. Такая структура позволяет наглядно представить логику взаимодействия объектов в рамках разрабатываемой системы.

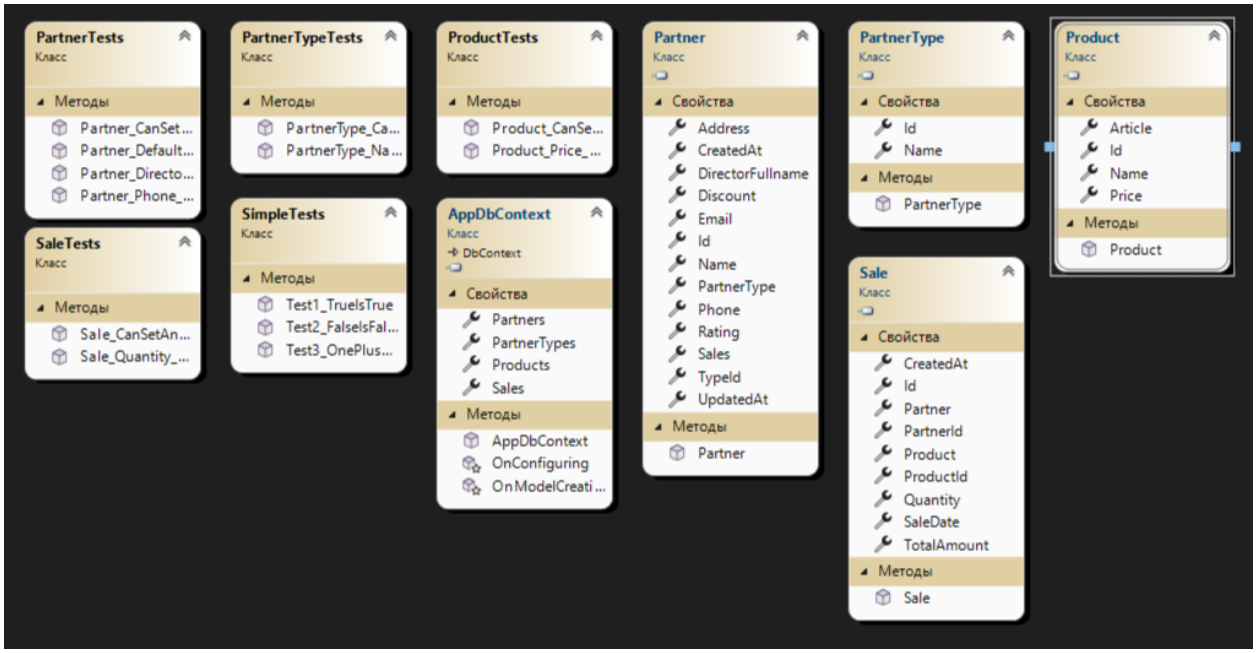

Рисунок 2 – Диаграмма классов программного продукта

```
{
    [Table("partners_poteha", Schema = "app")]
    Ссылок: 17
    public class Partner
    {
        [Key]
        [Column("id")]
        Ссылок: 6
        public int Id { get; set; }

        [Column("type_id")]
        Ссылок: 5
        public int TypeId { get; set; }

        [Required]
        [Column("name")]
        [MaxLength(200)]
        Ссылок: 7
        public string Name { get; set; }

        [Column("director_fullname")]
        [MaxLength(200)]
        Ссылок: 7
        public string DirectorFullname { get; set; }

        [Column("phone")]
        [MaxLength(20)]
        Ссылок: 7
        public string Phone { get; set; }

        [Column("email")]
        [MaxLength(100)]
        Ссылок: 4
        public string Email { get; set; }

        // ДОБАВЛЯЕМ ПОЛЕ АДРЕС
        [Column("address")]
        Ссылок: 3
        public string Address { get; set; }

        [Column("rating")]
        Ссылок: 7
        public int? Rating { get; set; }

        [Column("discount")]
        Ссылок: 6
        public int Discount { get; set; }

        [Column("created_at")]
        Ссылок: 5
        public DateTime CreatedAt { get; set; }

        [Column("updated_at")]
        Ссылок: 5
        public DateTime UpdatedAt { get; set; }

        Ссылок: 3
        public virtual PartnerType PartnerType { get; set; }
        Ссылок: 3
        public virtual ICollection<Sale> Sales { get; set; }
    }
}
```

Рисунок 3 - Код класса «Partner»

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Storage.ValueConversion;
using PotehaLibrary.Models;
using System;

namespace PotehaLibrary
{
    Ссылок: 6
    public class AppDbContext : DbContext
    {
        Ссылок: 3
        public DbSet<PartnerType> PartnerTypes { get; set; }
        Ссылок: 5
        public DbSet<Partner> Partners { get; set; }
        Ссылок: 2
        public DbSet<Product> Products { get; set; }
        Ссылок: 10
        public DbSet<Sale> Sales { get; set; }

        Ссылок: 0
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseNpgsql("Host=localhost;Database=poteha;Username=app2;Password=123456789;Encoding=UTF8;Search Path=app");
            optionsBuilder.EnableSensitiveDataLogging();
            optionsBuilder.EnableDetailedErrors();
        }

        Ссылок: 0
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.HasDefaultSchema("app");

            modelBuilder.Entity<PartnerType>().ToTable("partner_types_poteha");
            modelBuilder.Entity<Partner>().ToTable("partners_poteha");
            modelBuilder.Entity<Product>().ToTable("products_poteha");
            modelBuilder.Entity<Sale>().ToTable("sales_poteha");

            modelBuilder.Entity<PartnerType>(entity =>
            {
                entity.HasKey(e => e.Id);
                entity.Property(e => e.Id).HasColumnName("id");
                entity.Property(e => e.Name).HasColumnName("name").IsRequired().HasMaxLength(100);
            });

            modelBuilder.Entity<Partner>(entity =>
            {
                entity.HasKey(e => e.Id);
                entity.Property(e => e.Id).HasColumnName("id");
                entity.Property(e => e.TypeId).HasColumnName("type_id");
                entity.Property(e => e.Name).HasColumnName("name").IsRequired().HasMaxLength(200);
                entity.Property(e => e.DirectorFullname).HasColumnName("director_fullname").HasMaxLength(200);
                entity.Property(e => e.Phone).HasColumnName("phone").HasMaxLength(20);
                entity.Property(e => e.Email).HasColumnName("email").HasMaxLength(100);
                entity.Property(e => e.Rating).HasColumnName("rating");
                entity.Property(e => e.Discount).HasColumnName("discount");
                entity.Property(e => e.CreatedAt).HasColumnName("created_at");
                entity.Property(e => e.UpdatedAt).HasColumnName("updated_at");

                entity.HasOne(e => e.PartnerType)
                    .WithMany()
                    .HasForeignKey(e => e.TypeId)
                    .OnDelete(DeleteBehavior.Restrict);
```

Рисунок 4 - Часть контекста базы данных

```
using PotehaLibrary.Models;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace PotehaApp.ViewModels
{
    Ссылок: 9
    public class PartnerViewModel : INotifyPropertyChanged
    {
        private Partner _partner;

        Ссылок: 1
        public PartnerViewModel(Partner partner)
        {
            _partner = partner;
        }

        Ссылок: 10
        public int Id => _partner.Id;
        Ссылок: 5
        public string DisplayName => $"{_partner.PartnerType?.Name} / {_partner.Name}";
        Ссылок: 4
        public string Director => _partner.DirectorFullname ?? "Не указан";
        Ссылок: 4
        public string Phone => _partner.Phone ?? "Не указан";
        Ссылок: 7
        public int? Rating => _partner.Rating;
        Ссылок: 4
        public int Discount => _partner.Discount;

        Ссылок: 0
        public string Address => _partner.Address ?? "Не указан";

        Ссылок: 2
        public Partner Partner => _partner;

        public event PropertyChangedEventHandler PropertyChanged;
        Ссылок: 0
        protected void OnPropertyChanged([CallerMemberName] string name = null)
        {
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
        }
    }
}
```

Рисунок 5 - Часть программного сервиса «PartnerService»

Процесс создания программного модуля

Реализация программного модуля выполнялась согласно спецификациям, представленным в приложении 1. В ходе работы были созданы механизмы отображения перечня контрагентов, создания, корректировки и удаления записей, а также просмотра сведений о реализованной продукции. Разработанный модуль производит автоматическое вычисление персональной скидки партнера на основании суммарного объема закупок, обеспечивает выгрузку информации в текстовые форматы и обрабатывает исключительные ситуации с отображением понятных пользователю уведомлений.

Последовательность разработки

Работа над программным продуктом строилась из нескольких последовательных стадий:

– проектирование информационной структуры – создание логической модели данных, приведенной к третьей нормальной форме,

определение состава таблиц, установление связей и первичных ключей (графическое представление приведено на рисунке 2);

– формирование архитектуры решения – инициализация проекта на основе шаблона WPF Application с последующим подключением отдельной библиотеки классов, инкапсулирующей бизнес-логику;

– тестирование компонентов – выполнение модульных проверок и интеграционных испытаний для подтверждения корректности функционирования всех частей системы;

– подготовка документации – написание пояснительных комментариев к исходному коду, формирование описания возможностей приложения и руководства для конечного пользователя;

– обеспечение развертывания – создание установочных скриптов для базы данных и инструкций по инсталляции приложения на целевых машинах.

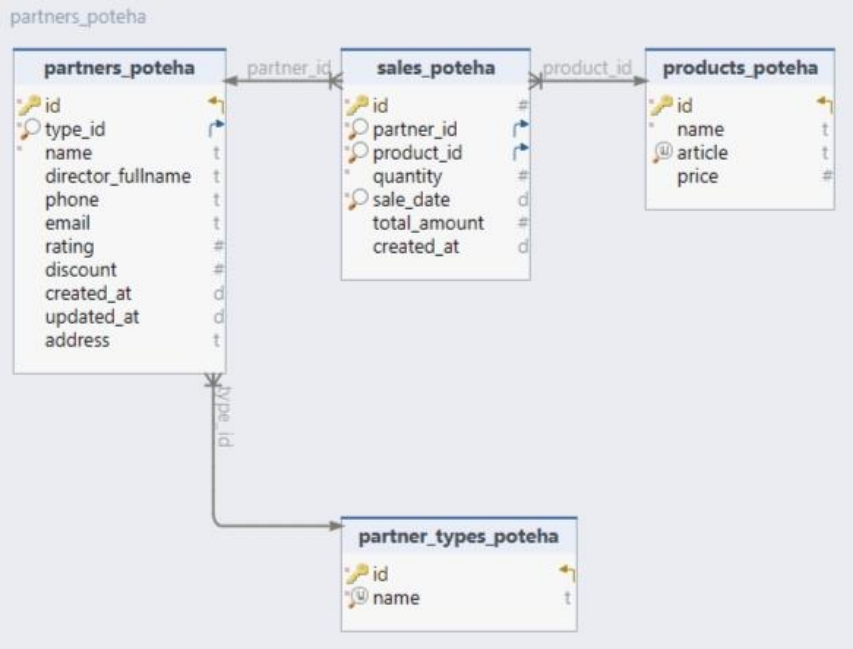

Рисунок 6 – Схема базы данных

Пользовательский интерфейс приложения

Для обеспечения взаимодействия с системой разработан набор экранных форм, обеспечивающих ввод, просмотр и модификацию данных:

– основное окно программы (рисунок 3) включает две функциональные области: в левой части отображается перечень контрагентов, в правой — информация о реализации продукции выбранным партнером. Переход между режимами работы осуществляется через главное меню и контекстное меню, вызываемое правой кнопкой мыши.

– форма редактирования сведений о партнере (рисунок 4) предназначена для создания новых записей и изменения существующих. На форме размещены элементы для указания наименования организации, выбора категории партнера из раскрывающегося списка, ввода целочисленного значения рейтинга, почтового адреса, контактных данных руководителя, телефонного номера и электронной почты.

– окно оформления продажи (рисунок 5) позволяет фиксировать факты реализации продукции. Пользователь выбирает наименование товара из справочника, указывает количество единиц и дату операции. Стоимость продажи рассчитывается программно на основании введенных данных и отображается в соответствующем поле формы.

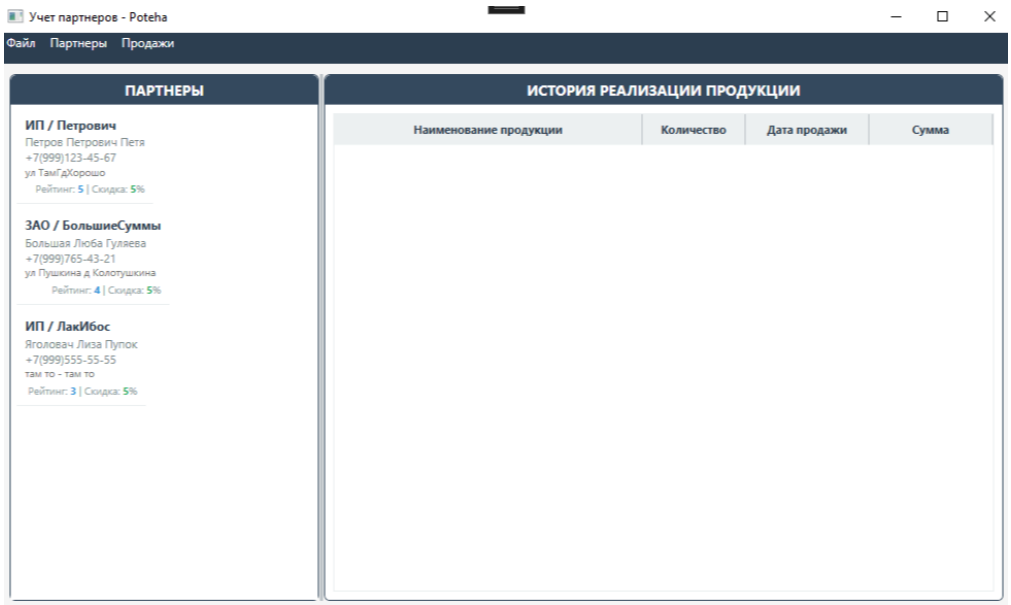

Рисунок 7 – Главное окно приложения

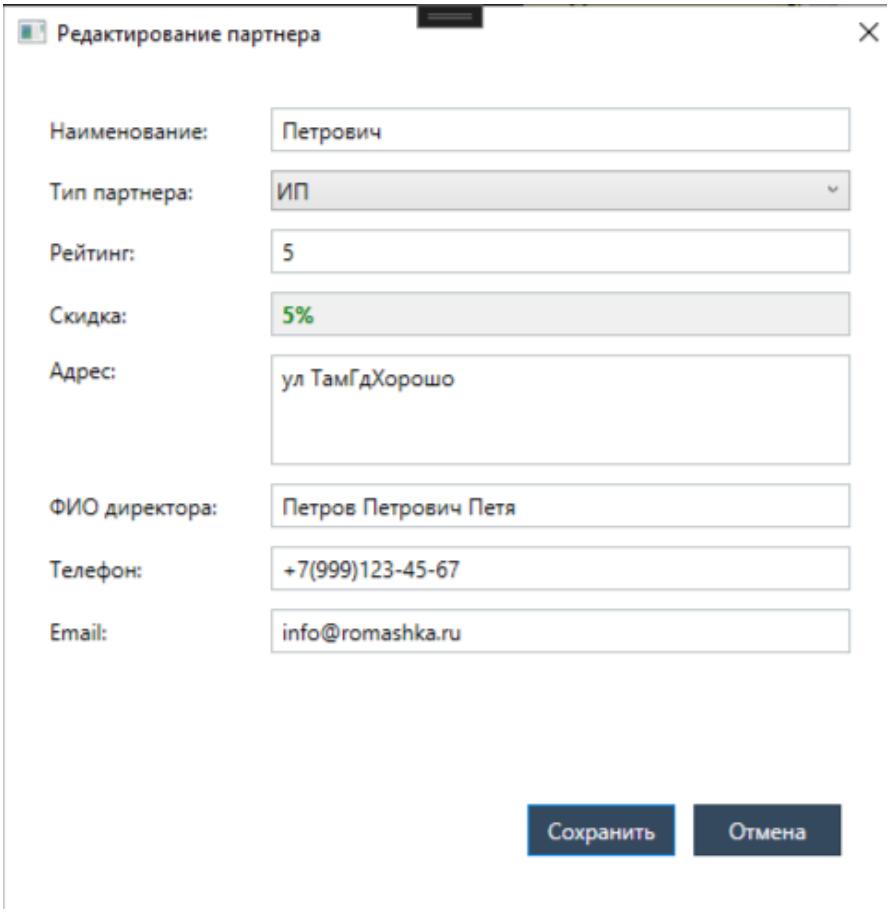

Рисунок 8 – Диалоговое окно добавления/редактирования партнера

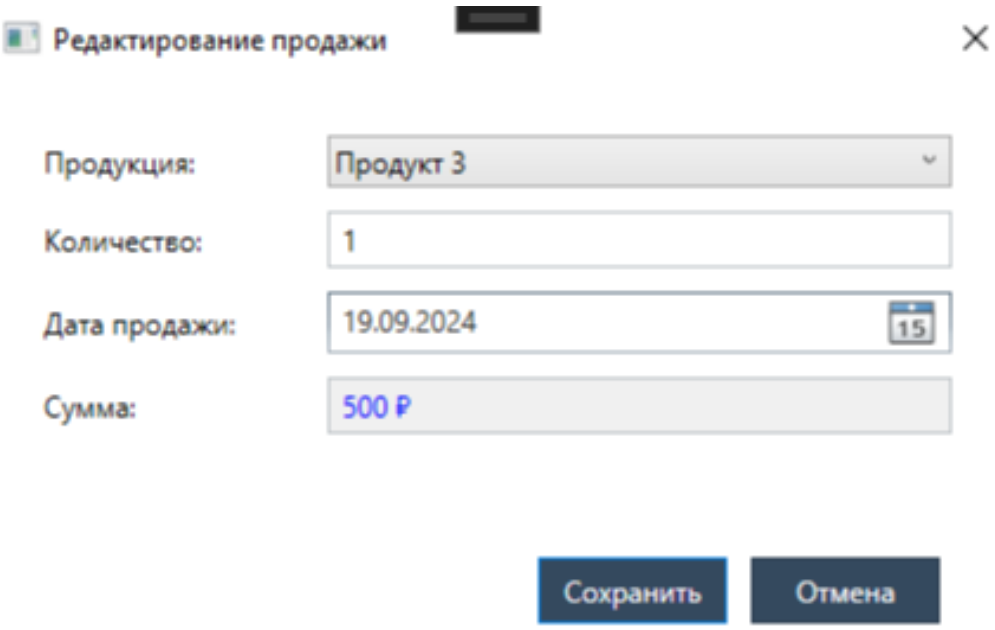

Рисунок 9 – Диалоговое окно добавления/редактирования продажи

Последовательность создания пользовательского интерфейса

Работы по формированию графической оболочки приложения проводились в несколько стадий:

Макетирование – на начальном этапе разрабатывались схематичные изображения будущих окон (пример макета приведен на рисунке 6). Определялось размещение управляющих элементов, структура основного окна с разделением на две информационные области, продумывалась система навигации с использованием главного и контекстного меню.

Визуальное оформление – на следующем этапе выполнялась настройка внешнего вида элементов. Для интерфейса выбрана сдержанная цветовая гамма в деловом стиле с преобладанием темно-синих и серых оттенков. Единообразие стилей обеспечивалось путем определения общих ресурсов оформления, что гарантировало одинаковый вид всех окон приложения (итоговый результат представлен на рисунке 3).

Реализация навигации – завершающим этапом стала программная реализация переходов между окнами и вызов контекстных меню для выполнения операций над выбранными записями.

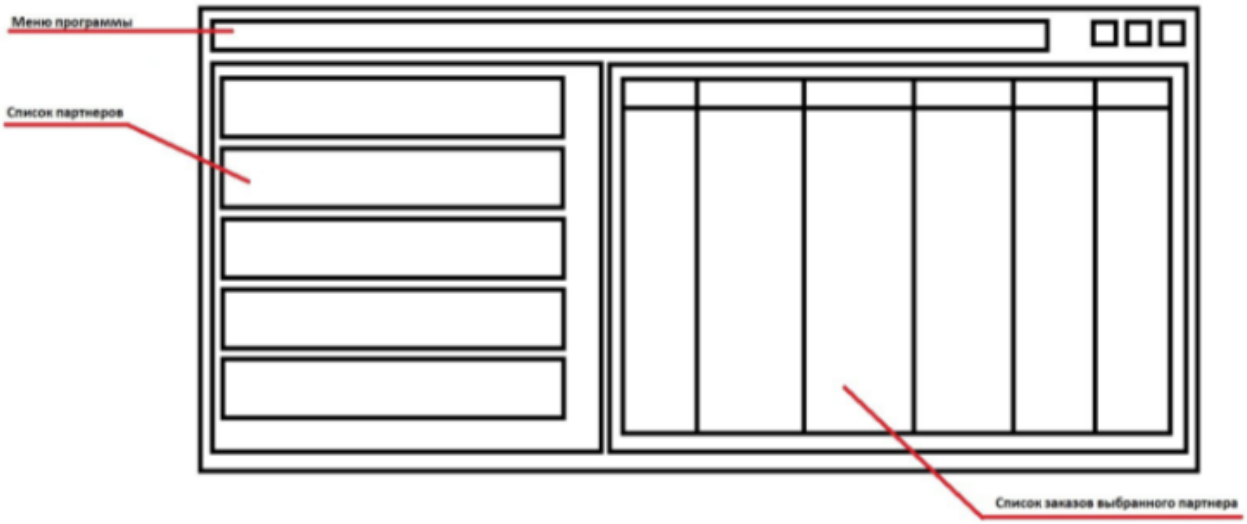

Рисунок 10 – Прототип интерфейса

3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества;

Обеспечение функциональности и качества программного кода

В процессе разработки применялся комплекс методов, направленных на достижение требуемого уровня функциональности и качества создаваемого программного продукта:

- Разделение ответственности – кодовая база структурирована по принципу модульности: выделены отдельные компоненты для работы с данными (библиотека классов), описания сущностей (модели) и связующей логики (ViewModel). Такой подход обеспечил возможность повторного использования компонентов и упростил процедуру тестирования.
- Архитектурный шаблон MVVM – применение паттерна Model-View-ViewModel позволило четко разграничить слои представления и бизнес-логики (схема взаимодействия компонентов показана на рисунке 7). Благодаря этому повысилась тестируемость отдельных частей приложения и упростилось внесение изменений в пользовательский интерфейс без модификации внутренней логики.
- Следование принципам SOLID – при построении иерархии классов учитывались требования единственной ответственности, расширяемости без модификации существующего кода и корректности наследования.
- Контроль вводимых данных – реализована автоматическая проверка значений, вводимых пользователем, на уровне моделей и элементов интерфейса, что позволило минимизировать ошибки при заполнении форм.
- Механизм обработки ошибок – для предотвращения аварийных завершений программы применялись блоки перехвата исключений с последующим информированием пользователя о возникших проблемах в понятной форме.

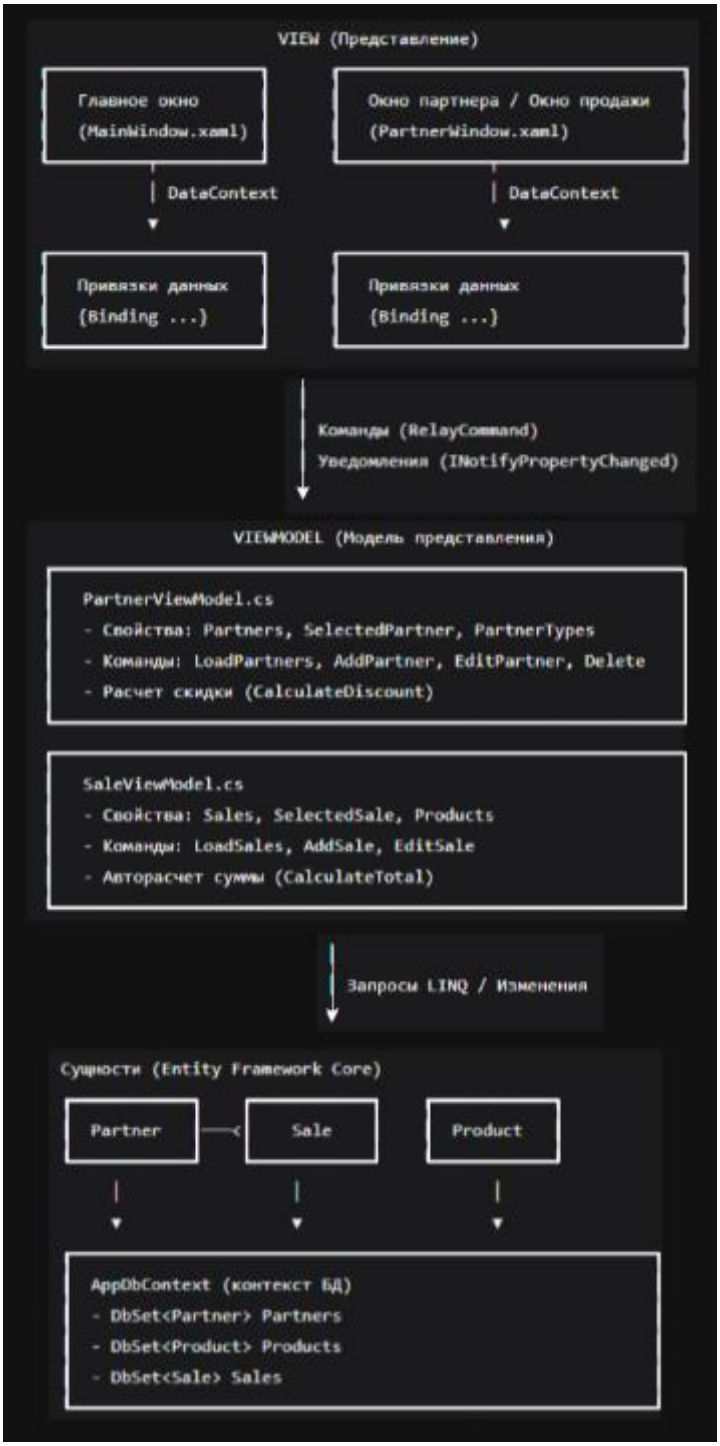

Рисунок 11 – Схема паттерна MVVM

3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;

Проверка программного кода на соответствие установленным требованиям

В ходе выполнения работ осуществлялся контроль качества исходного кода с использованием различных методов инспекции:

Применяемые подходы к проверке:

- Автоматизированный анализ – задействовались встроенные инструменты среды Visual Studio 2022, выполняющие непрерывную проверку синтаксиса и выявляющие потенциальные ошибки в процессе написания кода.
- Экспертная оценка – проводился разбор логической структуры компонентов с целью обнаружения скрытых дефектов и неоптимальных решений.
- Регламентированная проверка – осуществлялся контроль соблюдения принятых правил оформления и именования элементов программного кода.

Нормы оформления, принятые при разработке:

- Для именования классов и публичных свойств применялся стиль PascalCase;
- Переменные и методы именовались с использованием CamelCase;
- Наименования элементов выбирались таким образом, чтобы однозначно отражать их функциональное назначение;
- В сложных для понимания местах код снабжался пояснительными комментариями;
- Соблюдались единые правила форматирования, включая расстановку отступов и переносов.

Инструментарий, использованный для проверки:

- Штатные средства Visual Studio 2022, обеспечивающие подсветку синтаксиса и оперативное выявление некорректных конструкций;
- Встроенный анализатор управляемого кода, позволяющий обнаружить скрытые проблемы и отклонения от рекомендуемых практик;
- Функции автоматического рефакторинга и форматирования, применявшиеся для приведения кода к единому стандарту.

3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств;

Проведение отладки программного модуля с применением специализированных инструментов

Работы по выявлению и устранению ошибок в программном модуле выполнялись с использованием отладочных средств среды Visual Studio 2022. Процесс отладки включал следующие мероприятия:

- Расстановка контрольных точек – в ключевых участках кода устанавливались маркеры останова, позволяющие приостановить выполнение программы и изучить текущее состояние данных в момент остановки.
- Трассировка выполнения – применялось построчное исполнение кода с использованием клавиш F10 (обход вызываемых методов) и F11 (вход в вызываемые методы) для детального анализа последовательности операций.
- Мониторинг данных – в окнах Autos и Locals отслеживались актуальные значения переменных, что позволяло своевременно обнаружить некорректные или неожиданные изменения.
- Диагностика исключительных ситуаций – при возникновении ошибок исследовалась цепочка вызовов (stack trace) для точного определения места и причины сбоя.
- Интерактивная проверка – через окно Immediate выполнялись одиночные выражения и тестовые вызовы методов для оперативной проверки предположений о работе отдельных фрагментов кода без повторного запуска всего приложения.

3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения;

Разработка тестовых сценариев и наборов для проверки программного обеспечения

Организация процесса тестирования

Проверка работоспособности разработанного приложения осуществлялась на нескольких уровнях:

– Автономное тестирование компонентов – выполнялась изолированная проверка отдельных модулей программы без взаимодействия с другими частями системы;

– Проверка межмодульных связей – тестировалось корректное взаимодействие различных компонентов приложения между собой, а также их совместная работа с базой данных;

– Тестирование графического интерфейса – оценивалась правильность отображения элементов управления и реакции программы на действия пользователя.

Модульные тесты

Для автоматизированной проверки отдельных компонентов применялся фреймворк xUnit. В ходе работы были созданы тесты, охватывающие следующие аспекты:

– вычисление размера скидки для контрагента в зависимости от суммарного объема закупок;

– корректное поведение свойств в классах-моделях;

– правильность отображения данных и обработка пустых значений в промежуточном слое ViewModel.

На рисунке 8 представлены результаты прогона разработанных тестов. В числе проверяемых сценариев:

– отсутствие начисления скидки при нулевом объеме реализации;

– применение 5-процентной скидки при объеме продаж в диапазоне от 10 000 до 50 000 единиц; корректное преобразование наименования партнера для отображения в пользовательском интерфейсе.

```
using Xunit;
using PotehaLibrary.Models;
using System.Collections.Generic;

namespace PotehaTests
{
    Ссылок: 0
    public class PartnerTests
    {
        [Fact]
        Ссылок: 0
        public void Partner_DefaultDiscount_IsZero()
        {
            // Arrange
            var partner = new Partner();

            // Act
            var discount = partner.Discount;

            // Assert
            Assert.Equal(0, discount);
        }

        [Fact]
        Ссылок: 0
        public void Partner_CanSetAndGetProperties()
        {
            // Arrange
            var partner = new Partner();

            // Act
            partner.Id = 10;
            partner.Name = "Тестовый партнер";
            partner.Rating = 5;
            partner.Discount = 10;

            // Assert
            Assert.Equal(10, partner.Id);
            Assert.Equal("Тестовый партнер", partner.Name);
            Assert.Equal(5, partner.Rating);
            Assert.Equal(10, partner.Discount);
        }

        [Fact]
        Ссылок: 0
        public void Partner_Director_CanBeNull()
        {
            // Arrange
            var partner = new Partner();

            // Act
```

Рисунок 12 - Пример модульных тестов для работы с партнерами

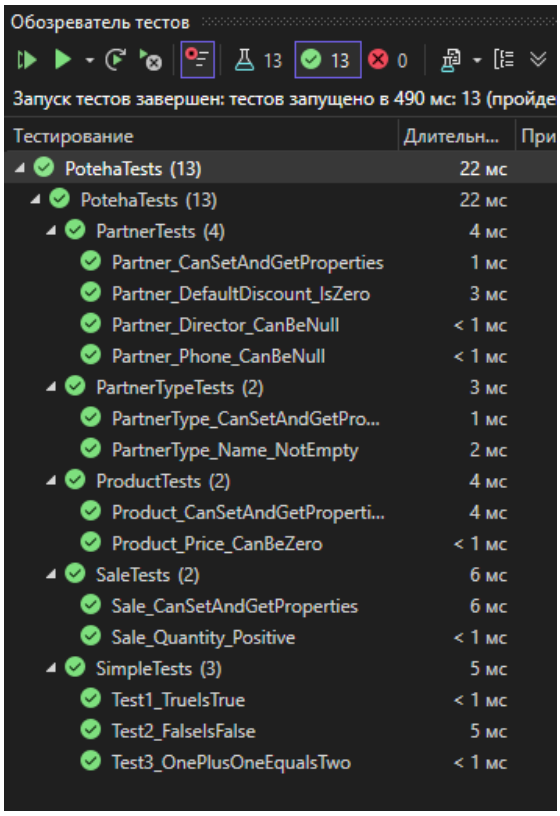

Рисунок 13 – Результаты выполнения unit-тестов

3.1.7. Интегрирование программных модулей;

Объединение программных компонентов в единую систему

Интеграция разработанных модулей выполнялась последовательно в несколько этапов. Изначально создавались два независимых компонента: модуль взаимодействия с базой данных, построенный на базе Entity Framework Core, и модуль бизнес-логики, оформленный в виде отдельной библиотеки классов. На следующем этапе эти компоненты связывались с модулем представления, реализованным с использованием WPF. Связующим звеном выступали классы ViewModel, которые обеспечивали передачу данных между интерфейсом и внутренней логикой приложения.

Подключение к базе данных реализовано через внедрение экземпляра контекста AppDbContext в конструкторы диалоговых окон, что позволило организовать централизованный доступ к данным для всех частей приложения. Для проверки корректности объединения компонентов выполнялись сквозные пользовательские сценарии, включающие регистрацию нового контрагента, оформление продажи и последующее обновление информации в интерфейсе, что подтвердило целостность работы всех модулей.

3.1.8. Верификация и аттестация программного обеспечения;

Проверка соответствия программного продукта установленным требованиям

Процедура верификации

Контроль корректности создаваемых компонентов осуществлялся на всех стадиях разработки. Использовались следующие способы проверки:

- Анализ исходного кода – оценивалось соблюдение принятых правил оформления и структурирования программного кода;
- Экспертиза проектных решений – рассматривалась обоснованность выбранной архитектуры и логики функционирования отдельных узлов;

– Запуск тестов – выполнялась проверка работоспособности с использованием подготовленных тестовых сценариев.

Процедура аттестации

По завершении работ проводилась итоговая оценка готовности программного продукта к эксплуатации. В ходе аттестации проверялись:

– полнота реализации функций, заявленных в техническом задании;

– безошибочное выполнение всех предусмотренных сценариев работы пользователя;

– корректная реакция программы на некорректные действия и внештатные ситуации;

– устойчивость функционирования при различных режимах работы.

По итогам аттестации установлено, что разработанное приложение полностью отвечает предъявленным требованиям и может быть допущено к применению в практической деятельности.

Заключение

За период прохождения практики с 9 по 17 марта 2026 года были углублены теоретические знания и усовершенствованы профессиональные навыки.

Достигнутые результаты:

- На основе изучения проектной и технической документации сформулированы требования к программным модулям, обеспечивающие их согласованное функционирование в общей структуре разрабатываемого ПО.
- Выполнено моделирование этапов разработки, позволившее упорядочить процесс объединения компонентов и выстроить логику их взаимодействия.
- Проведена отладка программных компонентов с применением профильных инструментов, что дало возможность обнаружить и исправить неполадки до завершения разработки.
- Созданы тестовые комплекты и сценарии для оценки работоспособности и устойчивости программного продукта.

Приобретенные практические навыки:

- объединение отдельных модулей в целостную программную систему;
- диагностика и исправление ошибок в коде с использованием отладочных средств;
- формирование тестовых наборов и сценариев для проверки ПО;
- анализ исходного кода на предмет соблюдения установленных правил оформления.

Список используемых источников

1. ГОСТ 19.102-77. Единая система программной документации. Стадии разработки. – М.: Стандартинформ, 2010. – 4 с.
2. ГОСТ 34.601-90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания. – М.: Стандартинформ, 2009. – 8 с.
3. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. – М.: Стандартинформ, 2011. – 104 с.
4. ГОСТ Р 51904-2002. Разработка программного обеспечения. Термины и определения. – М.: Госстандарт России, 2002. – 24 с.
5. Благодатских, В.А. Программная инженерия: учебник для вузов / В.А. Благодатских, В.А. Волнин, К.Ф. Поскакалов. – М.: Юрайт, 2022. – 415 с.
6. Терехов, А.А. Технология разработки программного обеспечения: учебное пособие / Под ред. А.А. Терехова. – М.: Форум, 2021. – 320 с.
7. Введение в Git [Электронный ресурс] / Git-scm.com. – Режим доступа: https://git-scm.com/book/ru/v2 (дата обращения: 13.03.2026).
8. Документация Microsoft Visual Studio 2022 [Электронный ресурс] / Microsoft Docs. – Режим доступа: https://docs.microsoft.com/ru-ru/visualstudio/ (дата обращения: 13.03.2026).
9. Руководство по C# и .NET [Электронный ресурс] / Metanit.com. – Режим доступа: https://metanit.com/sharp/tutorial/ (дата обращения: 14.03.2026).
10. WPF. Руководство для начинающих [Электронный ресурс] / Metanit.com. – Режим доступа: https://metanit.com/sharp/wpf/ (дата обращения: 14.03.2026).
11. Документация Entity Framework Core [Электронный ресурс] / Microsoft Docs. – Режим доступа: https://docs.microsoft.com/ru-ru/ef/core/ (дата обращения: 14.03.2026).

12. PostgreSQL. Документация [Электронный ресурс] / Postgrespro.ru. – Режим доступа: https://postgrespro.ru/docs/postgresql/ (дата обращения: 14.03.2026).

13. Паттерн MVVM. Описание и примеры [Электронный ресурс] / Metanit.com. – Режим доступа: https://metanit.com/sharp/wpf/23.php (дата обращения: 15.03.2026).

14. Модульное тестирование с использованием xUnit [Электронный ресурс] / Microsoft Docs. – Режим доступа: https://docs.microsoft.com/ru-ru/dotnet/core/testing/unit-testing-with-dotnet-test (дата обращения: 15.03.2026).

15. Обработка исключений в C# [Электронный ресурс] / Metanit.com. – Режим доступа: https://metanit.com/sharp/tutorial/2.15.php (дата обращения: 15.03.2026).

16. Принципы SOLID в объектно-ориентированном программировании [Электронный ресурс] / Habr.com. – Режим доступа: https://habr.com/ru/post/348286/ (дата обращения: 15.03.2026).

17. Методологии разработки программного обеспечения [Электронный ресурс] / Habr.com. – Режим доступа: https://habr.com/ru/post/495354/ (дата обращения: 12.03.2026).

18. Троелсен, Э. Язык программирования C# 9.0 и платформа .NET 5 / Э. Троелсен, Д. Джепикс. – СПб.: Питер, 2022. – 848 с.

19. Рихтер, Д. CLR via C#. Программирование на платформе Microsoft .NET Framework / Д. Рихтер. – СПб.: Питер, 2020. – 896 с.

Приложения

Приложение 1

Требования к программному обеспечению

Производственная компания «Мастер пол» выпускает напольные покрытия. Компания занимается производством и реализует свою продукцию через партнеров, которые доставляют продукцию компании до конечных потребителей.

Необходимо разработать подсистему для работы с партнерами компании, обеспечивающую следующий функционал:

- просмотр списка партнеров;
- добавление/редактирование данных о партнере;
- просмотр истории реализации продукции партнером.

При отображении истории реализации продукции должна выводиться следующая информация:

- наименование продукции;
- количество реализованной продукции;
- дата продажи.

При отображении и создании партнера должна использоваться следующая информация:

- наименование;
- тип партнера;
- рейтинг;
- адрес;
- ФИО директора;
- телефон;
- электронная почта.

Дополнительное требование: наличие возможности добавления/редактирования/удаления не только партнеров, но связанных с ними сущностей (тип партнера, продукция, история реализации продукции).

Исходя из предоставленных данных, для реализации системы необходимо в базе данных отобразить только четыре сущности:

– продукция;
– типы партнеров;
– партнеры;
– история реализации продукции партнерами.

Продукция представляет собой объект, у которого есть только артикул, который используется как уникальное значение для разграничения продукции между собой, и наименование. Так как в требованиях не указаны пункты о выводе другой информации, такой как стоимость продукции, изображение и прочее, то она не будет усмотрена в данной работе.

Тип партнера представляет собой объект, содержащий уникальное название и не более.

Партнер – комплексный объект, который кроме ранней представленной информации содержит и идентификатор своего типа.

История реализации продукции – объект, хранящий информацию о реализованной продукции, партнере, который реализовал продукцию, а также количество и дату продажи данной продукции.